Chapter 3

Optimisation convexe non différentiable
Non-smooth convex optimization

In this chapter, we are interested in the following mathematical program:

$$(P) : \min \ f(\boldsymbol{x}) \tag{3.1}$$

$$\text{s.t. } \boldsymbol{x} \in \mathcal{X} \tag{3.2}$$

where:

- $f : \mathbb{R}^n \to \mathbb{R}$ is a continuous, convex, not necessarily differentiable function,
- $\mathcal{X} \subset \mathbb{R}^n$ is nonempty, closed and convex,
- we assume that problem (P) has an optimal solution.

Since f continuous and $\mathcal{X}$ is closed, $\mathcal{X}$ bounded or $\lim_{||\boldsymbol{x}||\to\infty} = \infty$ are sufficient conditions for the existence of a global optimum to (P).

Example 3.0.1.

$$\min_{x \in \mathbb{R}} |x|$$

Example 3.0.2.

$$\min \; f(x, y) \tag{3.3}$$
$$-1 \leq x \leq 1 \tag{3.4}$$
$$-1 \leq y \leq 1 \tag{3.5}$$

where

$$f(x, y) = \max\{-x, x + y, x - 2y\}.$$

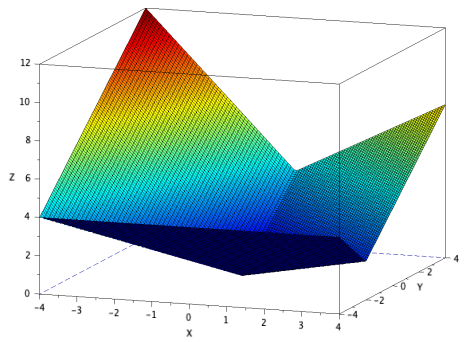


Figure 3.1: Surface of function f in example 3.0.2

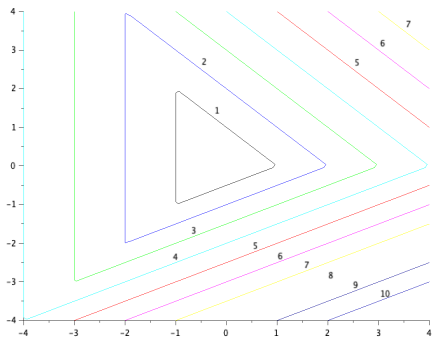


Figure 3.2: Contours of function f in example 3.0.2

3.1 Basic definitions

Subgradients extend the notion of gradient to convex, potentially non-differentiable functions.

Definition 3.1.1. Let $f_1 : \mathcal{X} \to \mathbb{R}$ be a convex function over $\mathcal{X}$. Vector $\boldsymbol{z} \in \mathbb{R}^n$ is a subgradient of f_1 at $\boldsymbol{x}_0$ if and only if

$$f_1(\boldsymbol{x}) - f_1(\boldsymbol{x}_0) \geq \boldsymbol{z}^\top(\boldsymbol{x} - \boldsymbol{x}_0) \quad \forall \boldsymbol{x} \in \mathbb{R}^n.$$

Let $f_2 : \mathcal{X} \to \mathbb{R}$ be a concave function over $\mathcal{X}$. Vector $\boldsymbol{z} \in \mathbb{R}^n$ is a subgradient of f_2 at $\boldsymbol{x}_0$ if and only if

$$f_2(\boldsymbol{x}) - f_2(\boldsymbol{x}_0) \leq \boldsymbol{z}^\top(\boldsymbol{x} - \boldsymbol{x}_0) \quad \forall \boldsymbol{x} \in \mathbb{R}^n.$$

Definition 3.1.2. The set of all subgradients of f at $\boldsymbol{x}_0$ is called subdifferential, and is denoted by $\partial f(\boldsymbol{x}_0)$.

This is a closed and convex set of $\mathbb{R}^n$.

We say that f is subdifferentiable at $\boldsymbol{x}_0$ if $\partial f(\boldsymbol{x}_0) \neq \emptyset$.

Theorem 3.1.1. *If $f : \mathcal{X} \to \mathbb{R}$ is convex or concave over $\mathcal{X}$, then f is subdifferentiable over* int $\mathcal{X}$.

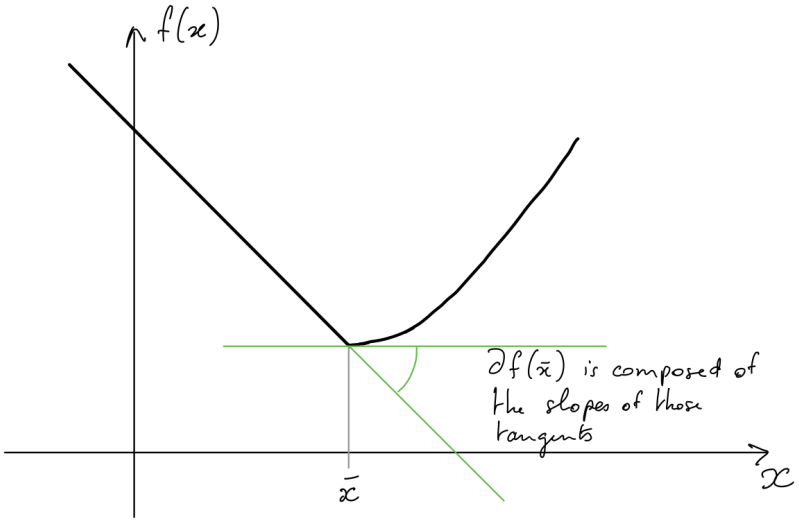


Figure 3.3: Illustration of the subdifferential

Definition 3.1.3 (Convex hull of a set)**.** The *convex hull* of a set $\mathcal{S} \subset \mathbb{R}^n$ is the set of all convex combinations (weighted averages) of its elements and is denoted by conv $\mathcal{S}$:

$$\text{conv}\,\mathcal{S} = \left\{ \boldsymbol{x} : \boldsymbol{x} = \sum_{j=1}^{k} \lambda_j \boldsymbol{x}^j, \sum_{j=1}^{k} \lambda_j = 1, \boldsymbol{\lambda} \in \mathbb{R}_+^k, \boldsymbol{x}^1, \ldots, \boldsymbol{x}^k \in \mathcal{S}, k \in \mathbb{N} \right\}.$$

Notation 3.1.4. In this document, we denote the set of all convex combination weights for k elements as Δ^k:

$$\Delta^k = \left\{ \boldsymbol{\lambda} \in \mathbb{R}_+^k : \sum_{j=1}^{k} \lambda_j = 1 \right\}.$$

The convex hull of $\mathcal{S}$ can then be defined as:

$$\text{conv}\,\mathcal{S} = \left\{ \boldsymbol{x} : \boldsymbol{x} = \sum_{j=1}^{k} \lambda_j \boldsymbol{x}^j, \boldsymbol{\lambda} \in \Delta^k, \boldsymbol{x}^1, \ldots, \boldsymbol{x}^k \in \mathcal{S}, k \in \mathbb{N} \right\}.$$

3.2 Typical use cases

In this section we present two classical convex nonsmooth optimization problems. For each of them, we derive its subdifferential, which will be helpful in interpreting and applying the numerical algorithms.

3.2.1 Minimize the pointwise maximum of convex smooth functions

$$(Minimax_1) : \min \left\{ \max \left\{ f_i(\boldsymbol{x}) : i \in \{1, \ldots, m\} \right\} : \boldsymbol{x} \in \mathcal{X} \right\}$$

with f_i convex and differentiable functions over $\mathcal{X}$.

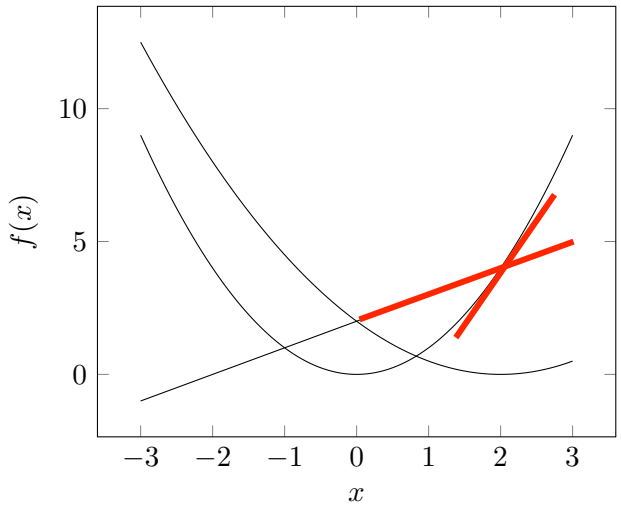
Example 3.2.1.

$$|x| = \max\{-x, x\}$$

Example 3.2.2.

$$(Minimax_2) : \min \quad f(x) := \max \left\{ x^2, \frac{1}{2}(x-2)^2, 2 + x \right\}$$
$$\textit{s.t. } x \in \mathbb{R}$$

Illustration of function $f(x)$



Intuitively, find $\partial f(-1)$ and $\partial f(2)$.
Show epi f.

Proposition 3.2.1. *Let f_i, $i \in \{1, \ldots, m\}$ be convex functions. Then $f(\boldsymbol{x}) = \max_i f_i(\boldsymbol{x})$ is convex.*

Proof. See below. □

Epigraph formulation

Definition 3.2.1. The epigraph of a function $f : \mathcal{X} \to \mathbb{R}$ is defined as

$$\text{epi } f = \big\{(\boldsymbol{x}, t) : \boldsymbol{x} \in \mathcal{X}, t \in \mathbb{R}, t \geq f(\boldsymbol{x})\big\}.$$

Proposition 3.2.2. *A function f is convex if and only if* epi$f$ *is a convex set.*

Proof. Straightforward. □

Proof of Proposition 3.2.1. The epigraph of function f is the intersection of the epigraphs of functions f_i. Hence, it is the intersection of convex sets and f is convex. □

In epigraph (mathematical programming) formulations, a variable t is introduced to model the value of a convex objective function:

$$(Epi) : \min\ t \tag{3.6}$$

$$\text{s.t. } t \geq f_i(\boldsymbol{x}) \qquad i \in \{1, \ldots, m\} \tag{3.7}$$

$$\boldsymbol{x} \in \mathcal{X}, t \in \mathbb{R} \tag{3.8}$$

Subdifferential

Proposition 3.2.3. *Let $\boldsymbol{x} \in \mathcal{X}$, and let us denote by $I(\boldsymbol{x}) = \arg\max\{f_i(\boldsymbol{x}) : i \in \{1, \ldots, m\}\}$ the set of indices of functions that achieve the value of f at $\boldsymbol{x}$. The subdifferential of f at $\boldsymbol{x}$ is:*

$$\partial f(\boldsymbol{x}) = \left\{ \sum_{k \in I(\boldsymbol{x})} \alpha_k \nabla f_k(\boldsymbol{x}) : \boldsymbol{\alpha} \in \Delta^{|I(\boldsymbol{x})|} \right\} = \text{conv } \{\nabla f_k(\boldsymbol{x}) : k \in I(\boldsymbol{x})\}$$

Proof. On blackboard. □

In particular, if there is a single function f_i such that $f(\boldsymbol{x}) = f_i(\boldsymbol{x})$, then f is differentiable at $\boldsymbol{x}$ and $\partial f(\boldsymbol{x}) = \{\nabla f(\boldsymbol{x})\}$.

Special case: maximum of affine functions When functions $f_i(x) = \boldsymbol{a}_i^\top x + b_i$ are affine, the subdifferential takes the following simple form:

$$\partial f(\boldsymbol{x}) = \left\{ \sum_{k \in I(\boldsymbol{x})} \alpha_k \boldsymbol{a}_k : \boldsymbol{\alpha} \in \Delta^{|I(\boldsymbol{x})|} \right\} = \text{conv } \{\boldsymbol{a}_k : k \in I(\boldsymbol{x})\}$$

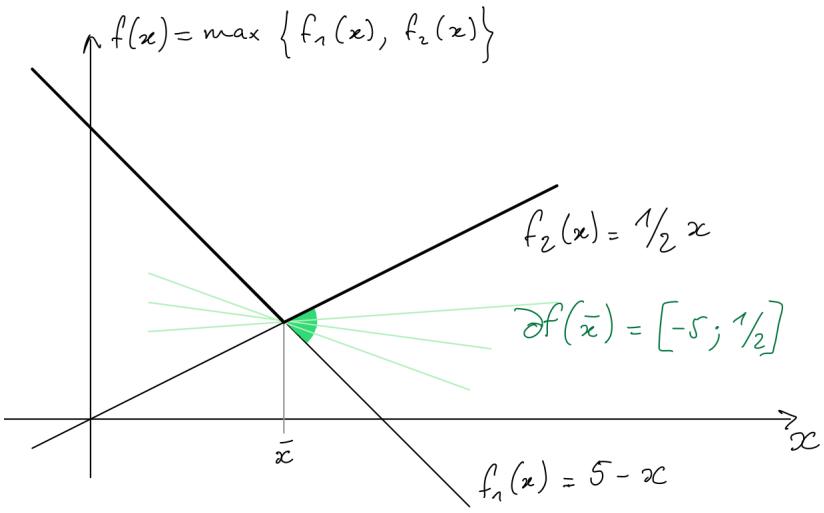


Figure 3.4: Illustration of the subdifferential for the pointwise maximum of affine functions

3.2.2 Lagrange duality

Lagrangian duality is a widely used concept in discrete optimization (notably in the field of integer linear programming). Solving the central optimization problem defined by this approach is an important motivation for convex nonsmooth optimization.

We consider mathematical programs of the following type:

$$(P_{Lag}) \ : \ \min \ f(\boldsymbol{x}) \tag{3.9}$$

$$\text{s.t. } \boldsymbol{g}(\boldsymbol{x}) \leq \boldsymbol{0} \tag{3.10}$$

$$\boldsymbol{x} \in X \tag{3.11}$$

with $X \subset \mathbb{R}^n$ and $\boldsymbol{g} = (g_i)_{i=1,\ldots,m}$, and $g_i : \mathbb{R}^n \to \mathbb{R}$ for $i = 1, \ldots, m$. Constraints $\boldsymbol{x} \in X$ usually define "simple" subproblems, while $\boldsymbol{g}(\boldsymbol{x}) \leq 0$ are "complicated constraints". In this context, removing the latter yields an optimization problem that is practically easy to solve.

Reminder

The Lagrangian relaxation consists in removing the complicated constraints and penalizing their violation using a penalty term in the objective function. This term is a weighted sum of the violations of individual constraints. Given fixed penalties (so-called Lagrange multipliers), solving the simple subproblems (with modified objective) yields a dual bound for the optimal value of the original problem. In practice, using Lagrangian relaxation involves finding a good vector of Lagrange multipliers in order to get a good dual bound. This optimization problem is called the Lagrangian dual problem.

Formally, the Lagrangian function $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^m_+ \to \mathbb{R}$ is the function defined by:

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) = f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x})$$

We define the Lagrangian dual function $L : \mathbb{R}^n \to \mathbb{R}$ as:

$$L(\boldsymbol{\lambda}) = \inf_{\boldsymbol{x} \in X} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda})$$

The Lagrangian dual problem is:

$$(D)_{Lag} : \sup_{\boldsymbol{\lambda} \in \mathbb{R}^m_+} L(\boldsymbol{\lambda})$$

Proposition 3.2.4. *The dual function L is concave.*

Example 3.2.3.

$$\min -x_1 - x_2 \tag{3.12}$$

$$\textit{s.t. } x_2 \leq \frac{3}{4}x_1 + \frac{7}{3} \tag{3.13}$$

$$x_2 \leq -5x_1 + \frac{45}{2} \tag{3.14}$$

$$x_2 \leq -\frac{1}{2}x_1 + 5 \tag{3.15}$$

$$x_2 \geq 6x_1 - 19 \tag{3.16}$$

$$x_1, x_2 \in \mathbb{N} \tag{3.17}$$

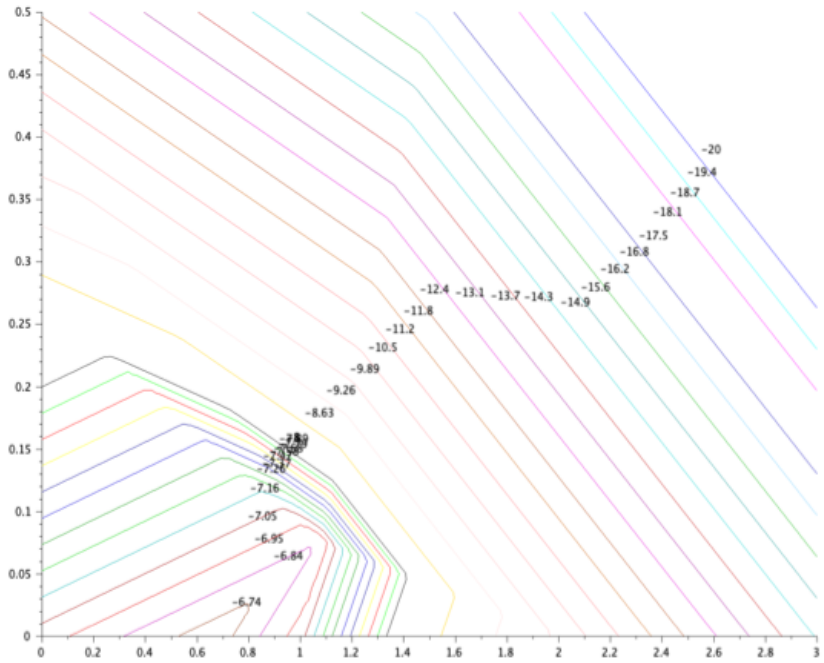


Figure 3.5: Illustration of the Lagrange dual function obtained by dualizing constraints (3.15) and (3.16).

Subgradients

Proposition 3.2.5. *For all* $\bar{\boldsymbol{\lambda}} \in \mathbb{R}_+^m$ *and* $\bar{\boldsymbol{x}} \in \arg\min_{\boldsymbol{x}\in\mathcal{X}} \mathcal{L}(\boldsymbol{x}, \bar{\boldsymbol{\lambda}})$*, we have* $\boldsymbol{g}(\bar{\boldsymbol{x}}) \in \partial L(\bar{\boldsymbol{\lambda}})$.

Proof. See below. □

Epigraph formulation

The Lagrangian dual problem can be formulated via an epigraph formulation: the Lagrangian dual function L is the pointwise minimum of (potentially infinitely many) affine functions.

More precisely:

$$\begin{aligned}
\max_{\boldsymbol{\lambda}\in\mathbb{R}_+^m} L(\boldsymbol{\lambda}) &= \max_{\boldsymbol{\lambda}\in\mathbb{R}_+^m} \min_{\boldsymbol{x}\in\mathcal{X}} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) \\
&= \max_{\boldsymbol{\lambda}\in\mathbb{R}_+^m} \min_{\boldsymbol{x}\in\mathcal{X}} f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x}) \\
&= \max\ t \\
&\qquad \text{s.t. } t \leq f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x}) \qquad \forall \boldsymbol{x} \in \mathcal{X} \\
&\qquad\qquad \boldsymbol{\lambda} \in \mathbb{R}_+^m, t \in \mathbb{R}
\end{aligned}$$

Remark that for a given $\boldsymbol{x} \in \mathcal{X}$, function $f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x})$ is affine, hence concave, in $\boldsymbol{\lambda}$.

This epigraph formulation allows the following simple proofs:

- The Lagrangian dual function L is the pointwise minimum of affine functions, so it is concave. Note that if $\mathcal{X}$ finite, L is piece-wise linear, and each piece corresponds to one element of $\mathcal{X}$.
- Let $\bar{\boldsymbol{\lambda}} \in \mathbb{R}_+^m$, and $I(\bar{\boldsymbol{\lambda}}) = \{\boldsymbol{x} \in \mathcal{X} : L(\bar{\boldsymbol{\lambda}}) = \mathcal{L}(\boldsymbol{x}, \bar{\boldsymbol{\lambda}})\}$. Then the subgradients of L at $\bar{\boldsymbol{\lambda}}$ are all convex combinations of $\nabla_{\boldsymbol{\lambda}}[f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x})]$ for $\boldsymbol{x} \in I(\bar{\boldsymbol{\lambda}})$, i.e.

$$\partial L(\bar{\boldsymbol{\lambda}}) = \text{conv}\{\boldsymbol{g}(\boldsymbol{x}) : \boldsymbol{x} \in I(\bar{\boldsymbol{\lambda}})\}.$$

3.3 Subgradient algorithms

3.3.1 Why do we need specific methods?

Bad luck: optimal solutions are often kinks.

Methods for nonsmooth optimization are closely related to the ones designed for optimizing differentiable functions. However, even if a function is "almost everywhere" differentiable, so that at first glance it looks like we could almost always compute gradients and use the same scheme, it is most of the time not the case.

Stopping criterion A typical stopping criterion for smooth optimization is $||\nabla f(\boldsymbol{x})|| \leq \epsilon$. However, this may never happen for nonsmooth functions, even as simple as absolute value.

Approximate subgradients One could be tempted to mimic the approximation of gradient using finite differences to compute approximate subgradients.

However, $\partial f(\boldsymbol{x})$ is not a continuous function of $\boldsymbol{x}$, so that difference quotients might not belong to the subdifferential, even in the limit.

Bibliography
The following section is based on the papers [FGG17], [BA18] and [BA00]. More information about the basics can be found in [BSS13] and [BHM98]. More advanced topics are covered in [Bon06].

3.3.2 The basic subgradient method

Notation
We want to solve the following problem:

$$(P) : \max\ L(\boldsymbol{\lambda}) \tag{3.18}$$

$$\text{s.t. } \boldsymbol{\lambda} \in \Omega \tag{3.19}$$

In the context of Lagrangian relaxation, L is the Lagrange dual function and Ω is typically $\mathbb{R}^m_+$. Hence, in the remainder, we will denote the dimension of Ω by m.

Algorithm The basic subgradient algorithm computes a sequence of feasible solutions based on the following recursion, inspired by the steepest descent method:

$$\boldsymbol{\lambda}^{k+1} = P_\Omega(\boldsymbol{\lambda}^k + s_k \boldsymbol{d}^k)$$

Here, s_k is the step length, and $\boldsymbol{d}^k \in \mathbb{R}^m$ is the moving direction, chosen in $\partial L(\boldsymbol{\lambda}^k)$. Function $P_\Omega : \mathbb{R}^n \to \Omega$ is a projection operator that ensures the feasibility of all iterates.

A number of subgradient-based optimization algorithms exist, according to the different choices made for these elements.

In the case of Lagrangian relaxation, this simple projection operator is typically used:

$$P_\Omega(\boldsymbol{\lambda}) = [\boldsymbol{\lambda}]^+ := (\max\{\lambda_i, 0\})_{i \in \{1,\ldots,m\}}$$

In the context of smooth optimization, the opposite of the gradient is always a descent direction. However, the opposite of subgradients (and more often extremal subgradients) are not always descent directions (see Figure 3.6).

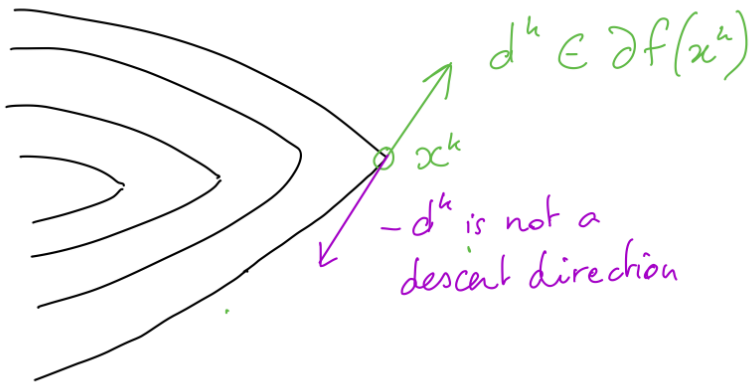


Figure 3.6: Subgradients do not always define descent directions.

Convergence Even if the direction chosen by the algorithm is not a descent direction, each iterate in the basic version of the algorithm gets closer (in terms of Euclidian norm) to all optimal solutions under an assumption on the step length.

Proposition 3.3.1. *Assume that $\boldsymbol{\lambda}^k$ is not optimal, $\boldsymbol{\lambda}^*$ is optimal and that the step length satisfies*

$$0 < s_k < \frac{2\left(L(\boldsymbol{\lambda}^*) - L(\boldsymbol{\lambda}^k)\right)}{||\boldsymbol{d}^k||^2}.$$

Then $||\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^*|| < ||\boldsymbol{\lambda}^k - \boldsymbol{\lambda}^*||$.

Proof. Idea on blackboard. □

In practice we obviously do not know $L(\boldsymbol{\lambda}^*)$. A basic workaround is to use the following rule for choosing s_k, that ensures convergence as well:

$$s_k = \beta_k \frac{UB - L(\boldsymbol{\lambda}^k)}{||\boldsymbol{d}^k||}.$$

Here:

- UB is an upper bound for (P),
- $0 < \beta_k < 2$ for all k,
- $\lim_{k\to\infty} \beta_k = 0$,
- $\lim_{k\to\infty} \sum_{l=1}^{\infty} \beta_l = \infty$

3.3.3 Deflected directions

The direction chosen at each iteration is usually based on a subgradient $\boldsymbol{g}^k \in \partial L(\boldsymbol{\lambda}^k)$, which is modified to achieve better practical convergence:

$$\boldsymbol{d}^k = \alpha_k \boldsymbol{g}^k + (1-\alpha_k)\boldsymbol{d}^{k-1}$$

with $\alpha_k \in [0,1]$ the so-called *deflection parameter*. The different variations described here are illustrated in Figure 3.7.

Basic subgradient The most basic subgradient method solely uses a subgradient at each step. In the above scheme, that corresponds to choosing $\alpha_k = 1$ for all k.

Details about the three following deflection directions can be found in [BA18].

Average direction strategy This strategy chooses the angle that bisects the current subgradient and the previous move direction:

$$\boldsymbol{d}^k_{ADS} = \boldsymbol{g}^k + \Psi^{ADS}_k \boldsymbol{d}^{k-1} \qquad \text{with } \Psi^{ADS}_k = \frac{||\boldsymbol{g}^k||}{||\boldsymbol{d}^{k-1}||}$$

Modified gradient technique This deflected direction strategy defines the move direction by:

$$\boldsymbol{d}^k_{MGT} = \boldsymbol{g}^k + \Psi^{MGT}_k \boldsymbol{d}^{k-1} \qquad \text{with } \Psi^{MGT}_k = \begin{cases} -\eta_k \frac{(\boldsymbol{g}^k)^\top \boldsymbol{d}^{k-1}}{||\boldsymbol{d}^{k-1}||^2} & \text{if } (\boldsymbol{g}^k)^\top \boldsymbol{d}^{k-1} < 0 \\ 0 & \text{otherwise} \end{cases}$$

where $0 < \eta_k \leq 2$.

Modified deflected subgradient This technique chooses a convex combination of d^k_{ADS} and d^k_{MGT} with the property that $\boldsymbol{d}^k$ forms an acute angle with both $\boldsymbol{d}^{k-1}$ and $\boldsymbol{\lambda}^* - \boldsymbol{\lambda}^k$, hence eliminating the zigzagging effect of the basic subgradient procedure [BA18].

$$\boldsymbol{d}^k_{MDS} = \boldsymbol{g}^k + \Psi^{MDS}_k \boldsymbol{d}^{k-1}$$

with

$$\Psi^{MGT}_k = \begin{cases} -\eta_k \frac{(1-\alpha_k)(\boldsymbol{g}^k)^\top \boldsymbol{d}^{k-1} + \alpha_k ||\boldsymbol{g}^k||\ ||\boldsymbol{d}^{k-1}||}{||\boldsymbol{d}^{k-1}||^2} & \text{if } (\boldsymbol{g}^k)^\top \boldsymbol{d}^{k-1} < 0 \\ 0 & \text{otherwise} \end{cases}$$

To enjoy the non-zigzagging property, parameters α_k and η_k can be chosen as:

$$\alpha_k = -\cos(\boldsymbol{g}^k, \boldsymbol{d}^{k-1}) = -\frac{(\boldsymbol{g}^k)^\top \boldsymbol{d}^{k-1}}{||\boldsymbol{g}^k||\ ||\boldsymbol{d}^{k-1}||} \text{ and } 0 < \eta_k \leq \frac{1}{2-\alpha_k}$$

Moreover, the Polyak rule (see next subsection) shall be used for the stepsize, with $0 < \beta_k < 1$.

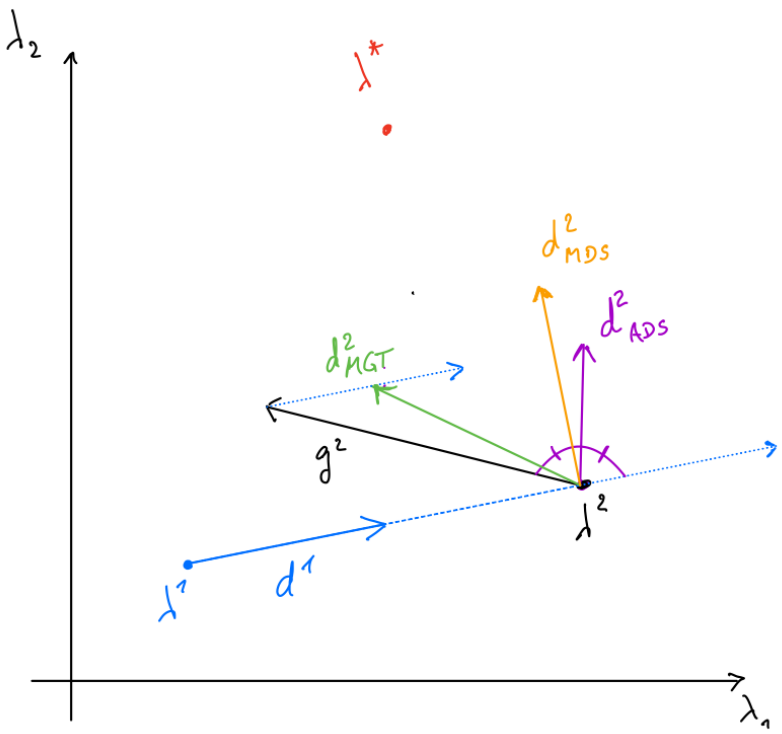


Figure 3.7: Illustration of the basic and modified directions

3.3.4 Stepsize rules

The step-size rule determines the value s_k used at each iteration.

Basic stepsizes

$$s_k = \frac{1}{k}, \text{ or } s_k = \rho s_k \text{ with } 0 < \rho < 1.$$

Polyak rule

$$s_k = \beta_k \frac{L_k^* - L(\boldsymbol{\lambda}^k)}{||\boldsymbol{d}^k||}.$$

Here, L_k^* can be either:

- the optimum of the problem $\max_{\boldsymbol{\lambda}} L(\boldsymbol{\lambda})$,
- an upper bound of the optimum,
- a "target value" $L(\hat{\boldsymbol{\lambda}}) + \delta_k$, where $\hat{\boldsymbol{\lambda}}$ is a feasible solution of the problem (so $L(\hat{\boldsymbol{\lambda}})$ might be a lower bound), and $\delta_k > 0$ is an surplus to ensure $s_k > 0$. We can choose either $\lim_{k\to\infty} \delta_k = 0$ or not. The same choices are possible for β_k.

ColorTV rule Let $\boldsymbol{g}_k$ be a subgradient of L at x_k, and $\rho > 0$ a fixed tolerance. Each iteration gets a color according to the following rule:

- $\boldsymbol{d}_k^\top \boldsymbol{g}_k > \rho$ and $L(\boldsymbol{\lambda}_k) > L(\boldsymbol{\lambda}_{k-1}) + \rho \max\{||L(\hat{\boldsymbol{\lambda}})||, 1\}$: green
- $\boldsymbol{d}_k^\top \boldsymbol{g}_k < \rho$ and $L(\boldsymbol{\lambda}_k) \geq L(\boldsymbol{\lambda}_{k-1})$: yellow *(sic)*
- otherwise: red

Each color c has a maximum number of consecutive iterations in that color n_c, and a multiplier m_c. If the number of consecutive iterations in color c is reached, then $\beta_k = m_c \beta_{k-1}$. Otherwise, $\beta_k = \beta_{k-1}$.

3.3.5 Stopping criterion

The algorithm ideally stops when $\mathbf{0} \in \partial L(\boldsymbol{\lambda}^k)$. In this case, $\boldsymbol{\lambda}^k$ is proved to be an optimal solution. However, this condition is most of the time hard to reach in practice, and anyway hard to detect, since $\partial L(\boldsymbol{\lambda}^k)$ is usually not known extensively: in most applications we only have access to an oracle that provides one single subgradient at a given iterate.

The stopping criteria often used in practice are the following:

- Maximum number of iterations reached.
- The best dual bound has not improved by more than $\epsilon_B\%$ in the last δ_B iterations.

- The gap between the best known value of L and the best dual bound has not decreased by more than $\epsilon_G\%$ in the last δ_G iterations.

3.4 Cutting plane algorithm

The methods described in Section 3.3 solve convex problems by iteratively generating a feasible point, a direction and a step size along this direction. This section presents another strategy that consists in iteratively solving more and more accurate approximations of the function to optimize.
We only present a basic algorithm, which can be improved in numerous ways to lead to very sophisticated and efficient methods.
We describe the approach for the specific case of the Lagrange dual problem. The adaptation in other contexts should be straightforward.

Let us recall the epigraph formulation of the Lagrange dual problem $\max_{\boldsymbol{\lambda}\in\mathbb{R}^m_+} L(\boldsymbol{\lambda})$ proposed in Section 3.2.2:

$$\begin{aligned}(D_{Epi}) : \max\ & t \\ \text{s.t. } & t \leq f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x}) \qquad \forall \boldsymbol{x} \in \mathcal{X} \\ & \boldsymbol{\lambda} \in \mathbb{R}^m_+, t \in \mathbb{R}.\end{aligned} \tag{3.20}$$

Since all constraints (3.20) are affine in the decision variables $(\boldsymbol{\lambda}, t)$, this model is a linear program that can be solved using, for example, the simplex algorithm. Unfortunately, such an approach is most of the time not viable, because the number of elements in $\mathcal{X}$ is usually very large, or even infinite. Moreover, in the context of mixed-integer linear programming, $\mathcal{X}$ might be the set of feasible solutions of a combinatorial problem, which is not explicitly known.

For these reasons, instead of solving (D_{Epi}) directly, we propose to use a cutting plane algorithm, which consists in iteratively adding constraints (3.20) when they are needed. More precisely, suppose that we know a subset $\{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^r\} \subseteq \mathcal{X}$, and let us consider the following approximation problem:

$$\begin{aligned}(D_{Epi})^r : \max\ & t \\ \text{s.t. } & t \leq f(\boldsymbol{x}^k) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x}^k) \qquad \forall k \in \{1, \ldots, r\} \\ & \boldsymbol{\lambda} \in \mathbb{R}^m_+, t \in \mathbb{R}.\end{aligned} \tag{3.21}$$

Problem $(D_{Epi})^r$ is often called the **master program**, and is obviously a relaxation of (D_{Epi}), which can be solved as any linear program. Let us suppose that, at iteration r of the algorithm, an optimal solution $(\boldsymbol{\lambda}^r, t^r)$ of $(D_{Epi})^r$ has been determined. We remark that, if $(\boldsymbol{\lambda}^r, t^r)$ is also a feasible solution to the initial problem (D_{Epi}), then it is an optimal solution for it. Indeed, since $(D_{Epi})^r$ is

a relaxation of (D_{Epi}), we have $Opt((D_{Epi})) \leq Opt((D_{Epi})^r) = t^r$. Moreover, since $(\boldsymbol{\lambda}^r, t^r)$ is feasible for $Opt((D_{Epi}))$, we also have $Opt((D_{Epi})) \geq t^r$. So $Opt((D_{Epi})) = t^r$.

In order to check whether $(\boldsymbol{\lambda}^r, t^r)$ is feasible for (D_{Epi}), we need to verify if it satisfies all Constraints (3.20), *i.e.*:

$$t^r \leq f(\boldsymbol{x}) + (\boldsymbol{\lambda}^r)^\top \boldsymbol{g}(\boldsymbol{x}) \qquad \forall \boldsymbol{x} \in \mathcal{X}.$$

This is equivalent to:

$$t^r \leq \min \left\{ f(\boldsymbol{x}) + (\boldsymbol{\lambda}^r)^\top \boldsymbol{g}(\boldsymbol{x}) : \boldsymbol{x} \in \mathcal{X} \right\}.$$

The **separation problem** consists in determining whether this inequality is satisfied or not and, if not, exhibiting a solution $\boldsymbol{x} \in \mathcal{X}$ that violates Constraint (3.20). We remark that the separation problem comes exactly to evaluating the dual function, by solving the Lagrange sub-problem:

$$\begin{aligned} (Sep(\bar{\boldsymbol{\lambda}})) : \min \ & L(\bar{\boldsymbol{\lambda}}) = f(\boldsymbol{x}) + \bar{\boldsymbol{\lambda}}^\top g(\boldsymbol{x}) \\ & \text{s.t. } \boldsymbol{x} \in \mathcal{X}. \end{aligned}$$

3.4.1 The basic cutting plane algorithm

Algorithm 3.4.1 describes the basic procedure to solve the Lagrange dual problem with help of a cutting plane method.

> We note that Algorithm 3.4.1 only provides an optimal solution to the dual problem, and not for the primal problem. For the special case where the primal problem is convex, we will also show how a primal solution can be obtained.

The algorithm updates an upper and a lower bound at each iteration. The upper bound UB is simply the optimal value of current relaxation. We remark that , since the set of constraints at a given iteration contains the constraints of the previous iterations, the values of t^r are non-increasing. At any iteration r, vector $\boldsymbol{\lambda}^r$ is a feasible solution of the dual problem. Hence, the lower bound LB is defined at `line 9` as the best of those solutions obtained throughout the execution of the algorithm. Note that $L(\boldsymbol{\lambda}^r)$ is not monotone in general.

3.4.2 Obtaining a primal solution

Let us consider the special case where the primal problem (P_{Lag}) is a convex mathematical program: functions f and g_i, $i \in \{1, \ldots, m\}$ are convex, and $\mathcal{X}$ is a convex set. We assume that we know a set $\{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^r\} \subseteq \mathcal{X}$ of feasible solutions of the Lagrange subproblems. Typically, this is the set of solutions obtained during the course of Algorithm 3.4.1, at `line 7`.

Algorithm 3.4.1: Basic cutting plane algorithm.
Input: Problem $(D)_{Lag}$, optimality tolerance $\epsilon \geq 0$
Output: A (near-)optimal solution of $(D)_{Lag}$

Initialization
Find a feasible primal solution: $\boldsymbol{x}^1 \in \mathcal{X}$ such that $\boldsymbol{g}(\boldsymbol{x}^1) \leq \mathbf{0}$.
$r \leftarrow 2$, $LB \leftarrow -\infty$, $UB \leftarrow \infty$.
Main loop
while $UB - LB > \epsilon$ **do**
  Solve the master problem $(D)^{r-1}_{Epi}$ and let $(\boldsymbol{\lambda}^r, t^r)$ be an optimal solution.
  Compute $L(\boldsymbol{\lambda}^r)$ by solving the separation problem $Sep(\boldsymbol{\lambda}^r)$, and let $\bar{\boldsymbol{x}}^r$ be an optimal solution.
  Update UB: $UB \leftarrow t^r$.
  Update LB: $LB \leftarrow \max\{LB, L(\boldsymbol{\lambda}^r)\}$.
  if $UB - LB > \epsilon$ **then**
    Define $(D)^r_{Epi}$ by adding constraint $t \leq f(\boldsymbol{x}^r) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x}^r)$ to $(D)^{r-1}_{Epi}$.
    $r \leftarrow r + 1$

Proposition (3.4.1) below shows that any feasible solution of the following linear program (with decision variables $\boldsymbol{\lambda}$) yields a feasible solution of (P_{Lag}).

$$
\begin{aligned}
(H)^r : \min \quad & \sum_{k=1}^{r} f(\boldsymbol{x}^k)\mu_k \\
\text{s.t.} \quad & \sum_{k=1}^{r} \boldsymbol{g}(\boldsymbol{x}^k)\mu_k \leq \mathbf{0} \\
& \sum_{k=1}^{r} \mu_k = 1 \\
& \mu \in \mathbb{R}^r_+
\end{aligned}
$$

Proposition 3.4.1. *Let us assume that the primal problem (P_{Lag}) is a convex mathematical program: functions f and g_i, $i \in \{1, \ldots, m\}$ are convex, and $\mathcal{X}$ is a convex set. Besides, let $\{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^r\} \subseteq \mathcal{X}$. Consider a feasible solution $\boldsymbol{\mu}$ of $(H)^r$, and define $\bar{\boldsymbol{x}} = \sum_{k=1}^{r} \boldsymbol{x}^k \mu_k$.*

Then $\bar{\boldsymbol{x}}$ is a feasible solution of (P_{Lag}). Moreover, the objective value of $\boldsymbol{\mu}$ in $(H)^r$ is an upper bound on the cost of $\bar{\boldsymbol{x}}$ in (P_{Lag}): $f(\bar{\boldsymbol{x}}) \leq \sum_{k=1}^{r} f(\boldsymbol{x}^k)\mu_k$.

Proof. The result directly comes from convexity arguments: $\bar{\boldsymbol{x}}$ is a convex combination of $\boldsymbol{x}^k$, $k \in \{1, \ldots, r\}$.

- Since $\boldsymbol{x}^k \in \mathcal{X}$ for all $k \in \{1, \ldots, r\}$ and $\mathcal{X}$ is convex, $\sum_{k=1}^{r} \boldsymbol{x}^k \mu_k \in \mathcal{X}$.
- Since function g_i, $i \in \{1, \ldots, m\}$ is convex, $g_i(\sum_{k=1}^{r} \boldsymbol{x}^k \mu_k) \leq \sum_{k=1}^{r} g_i(\boldsymbol{x}^k)\mu_k \leq 0$, the rightmost inequality resulting from the feasibility of $\boldsymbol{\mu}$ for $(H)^r$.

This proves that $\bar{\boldsymbol{x}}$ is feasible for (P_{Lag}). The bound over $f(\bar{\boldsymbol{x}})$ is derived using the same arguments. $\square$

In the context of Algorithm 3.4.1, it is not necessary to solve $(H)^r$ to obtain its optimal solution. Indeed, the reader might have noticed that $(H)^r$ is the dual problem (in the sense of linear programming) of problem (D^r_{Lag}). Hence, optimal variables $\boldsymbol{\mu}$ are equal to the optimal dual variables associated with constraints (3.21), computed at `line 6`.

Bibliography

[BA00] Francisco Barahona and Ranga Anbil. The volume algorithm: producing primal solutions with a subgradient method. *Mathematical Programming*, 87(3):385–399, May 2000.

[BA18] Rachid Belgacem and Abdessamad Amir. A new modified deflected subgradient method. *Journal of King Saud University - Science*, 30(4):561–567, October 2018.

[BHM98] Dimitri P Bertsekas, WW Hager, and OL Mangasarian. *Nonlinear programming*. Athena Scientific Belmont, MA, 1998.

[Bon06] Introduction to Nonsmooth Optimization. In J. Frédéric Bonnans, J. Charles Gilbert, Claude Lemaréchal, and Claudia A. Sagastizabal, editors, *Numerical Optimization: Theoretical and Practical Aspects*, pages 109–118. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.

[BSS13] Mokhtar S. Bazaraa, Hanif D. Sherali, and C. M. Shetty. *Nonlinear Programming: Theory and Algorithms*. John Wiley & Sons, June 2013.

[FGG17] Antonio Frangioni, Bernard Gendron, and Enrico Gorgone. On the computational efficiency of subgradient methods: a case study with lagrangian bounds. *Mathematical Programming Computation*, 9(4):573–604, 2017.